

APIs für die AI Nutzung: Wie geht das?

Software Architecture Forum 2026

Thilo Frotscher & Erik Wilde

16. Juni 2026



This work is licensed under a CC
Attribution 4.0 International License

Outline

1. [Introduction](#) [3]
2. [API Design & Description in the Age of AI](#) [6]
3. [Exposing API Landscapes for AI Consumption](#) [9]
4. [MCP to the Rescue!?!](#) [2]
5. [Conclusions](#) [2]



Thilo Frotscher

- Kurator & Trainer [iSAQB Modul API](#)
- Independent Expert for APIs & Integration Architectures
- Lead Developer / Technical Architect / Trainer
- Author & Speaker
- Advisory Board Member for [API Conference](#) & [betterCode\(\) API](#)

APICONFERENCE

betterCode()



Erik Wilde

- Kurator & Trainer [iSAQB Modul API](#)
- Head of Enterprise Strategy at [Jentic](#)
- Ambassador for the [OpenAPI Initiative \(OAI\)](#)
- [LinkedIn](#) [linkedin.com/in/erikwilde](https://www.linkedin.com/in/erikwilde)
- [YouTube](#) [youtube.com/ErikWilde](https://www.youtube.com/ErikWilde)



iSAQB Modul Application Programming Interfaces (API)

(1) Warum APIs wichtig sind

(5) Beschreibung von APIs

(2) Wie APIs Wert erzeugen

(6) API Lifecycle & API Tooling

(3) API Stile & Technologien

(7) API Security

(4) API-Design

(8) APIs at Scale: Plattformen & Governance



Outline

1. [Introduction](#) [3]
2. [API Design & Description in the Age of AI](#) [6]
3. [Exposing API Landscapes for AI Consumption](#) [9]
4. [MCP to the Rescue!?!](#) [2]
5. [Conclusions](#) [2]



Creating APIs (in the old days...) #1

- Requirements Analysis
 - Talk to your consumers
 - Outside-in approach
- API First
 - Imagine you are on a greenfield
 - Do not create APIs based on existing systems or use cases



Creating APIs (in the old days...) #2

■ API Design

- Use iterative approach of design – review – design
- Implement good practices for API Design
- Make sure to have API Design Guidelines in your organization
- Focus on usability, consistency, comprehensibility, extensibility
- Use naming conventions and consistent endpoint structures
- Favor generic APIs over support for specific consumers / use cases



Creating APIs (in the old days...) #3

- API specification
 - Use standard description languages, like OpenAPI or AsyncAPI
 - Make sure to document as many aspects as possible, including all response status codes, parameters and attributes



Creating APIs (in the old days...) #4

- API documentation
 - Technical details missing in the API specification
 - Information explaining how endpoints work
 - Semantics of data structures and runtime behaviour
 - SDKs, quick-start guides



Creating APIs (in the old days...) #5

- Make APIs discoverable

- Register APIs in dev portals, API marketplaces, or internal catalogs
- Provide metadata, tags and standardized authentication details
- Offer sandbox environments and test credentials
- Maintain backwards compatibility



Today: AI Everywhere

- AI Coding Agents generate code that consumes APIs
- AI Business Agents discover and consume APIs autonomously
- What does all this mean for API providers?
 - What changes, what stays the same?



Outline

1. [Introduction](#) [3]
2. [API Design & Description in the Age of AI](#) [6]
3. [Exposing API Landscapes for AI Consumption](#) [9]
4. [MCP to the Rescue!?!](#) [2]
5. [Conclusions](#) [2]



AI Integration Is the New Moat

AI Integration Is the New Moat



By [Tim O'Reilly](#)

October 27, 2025 • 6 minute read



Share

The electrical system warning light had gone on in my Kona EV over the weekend, and all the manual said was to take it to the dealer for evaluation. I first tried scheduling an appointment via the website, and it reminded me how the web, once a marvel, is looking awfully clunky these days. There were lots of options for services to schedule, but it wasn't at all clear which of them I might want.



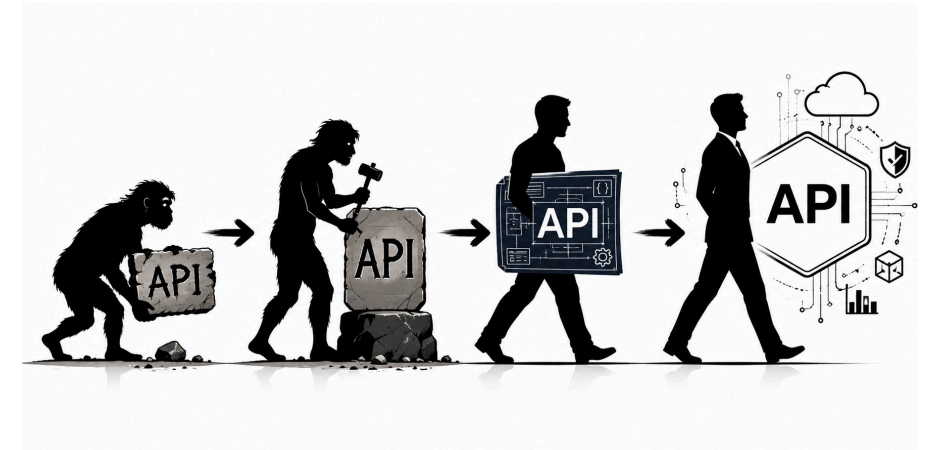
When LLMs Read Your APIs

- APIs were typically consumed by developers
- Developers write code that *often fills DX gaps*
- LLMs have more limited context than developers
 - *Coding agents* shouldn't depend on developers selecting APIs
 - *Business agents* need to fully “understand” APIs



Four Levels of API Evolution

1. *Improve descriptions* → easy, no code change, immediately more agent-friendly
2. *Improve definitions (backwards-compatible)* → change the API, don't break consumers
3. *Revise the API* → new API; consumers must migrate
4. *Add workflows on top* → orchestration above existing APIs



Level 1a: Do Your DX Homework

- Most API descriptions need quality improvements anyway
- Better descriptions benefit *everyone*, not just AI agents
 - Complete and meaningful operation summaries
 - Comprehensive examples for all operations
 - Clear error handling
 - Consistent terminology and naming
- Good DX is the foundation for good AX



Level 1b: Now There's AX Homework, Too

- AI agents need information developers don't typically require:
 - Everything that developers usually “just know”
 - Treat every API like an external one for developers with no context
- OpenAPI Initiative (OAI) might add AX-specific fields
- Until then, leverage description fields strategically



Level 2: Backwards-Compatible Evolution

- Improve the API itself *without* breaking existing consumers
 - Add optional fields and parameters
 - Enrich responses with new optional structures
 - Add new endpoints alongside existing ones
 - Tighten constraints; never loosen contracts
- Works well in well-governed API landscapes
- Morphs into *Level 3* in brittle or ungoverned API landscapes



Level 4: (Arazzo) Workflows

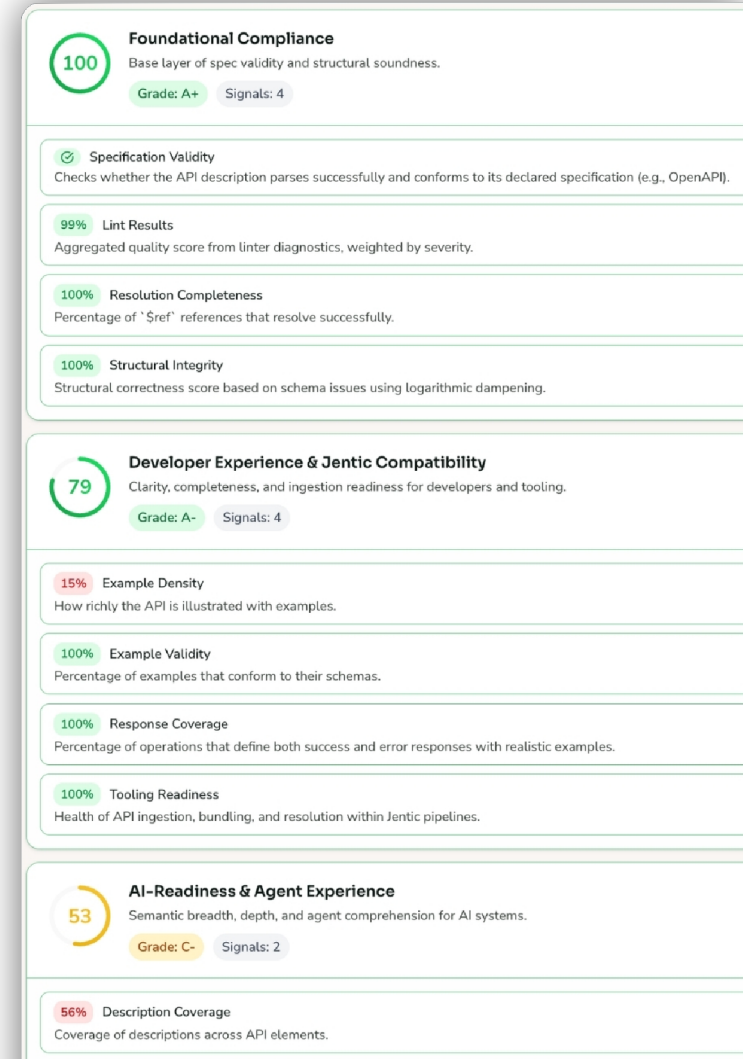
- *Declarative*: define the data flow and success criteria
- OpenAPI Initiative (OAI) standard for API workflows
- References operationIds from OpenAPI/AsyncAPI documents



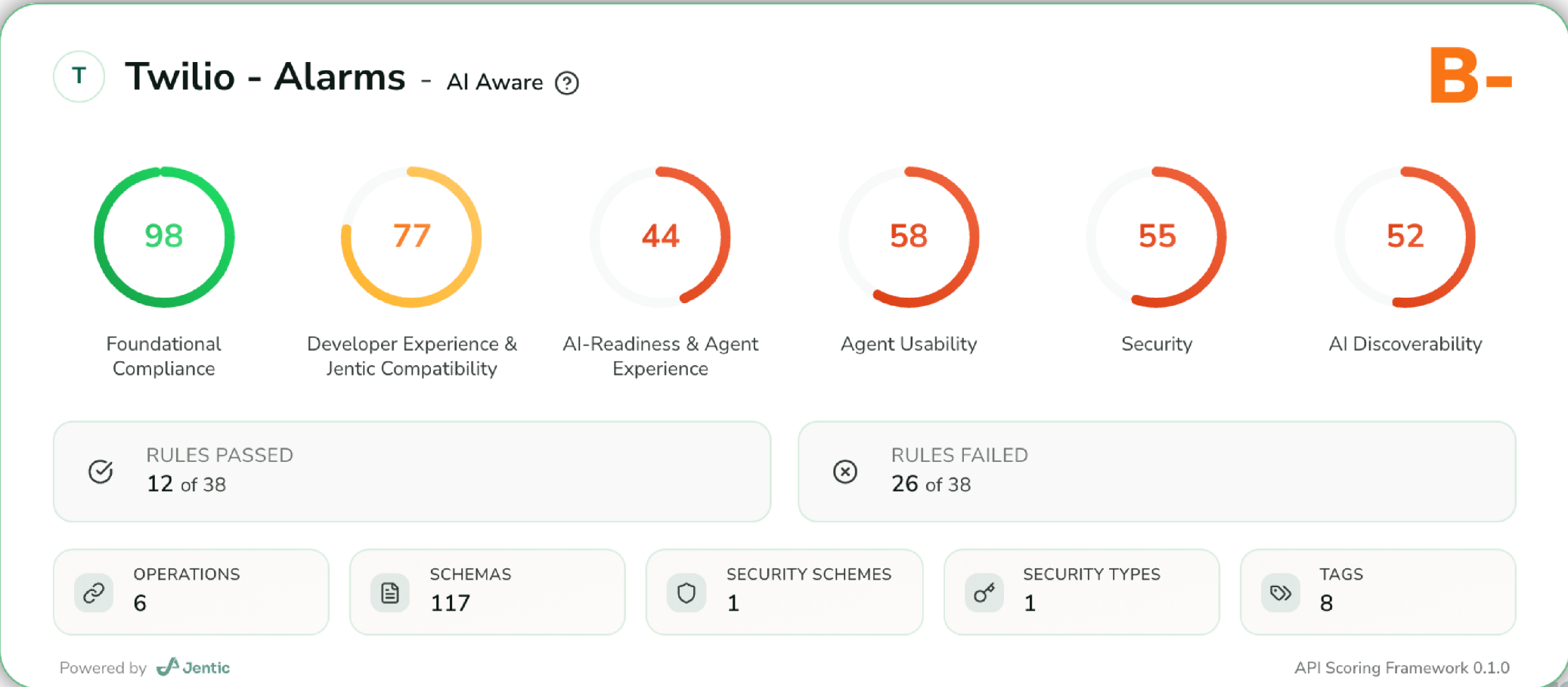
```
workflows:
  - workflowId: weekend-getaway
    steps:
      - stepId: listHotels
        description: Retrieve hotels available in the destination city
        operationPath: '{$sourceDescriptions.liteapi-hotel-
data.url}#/paths/~1data~1hotels/get'
        parameters:
          - name: cityName
            in: query
            value: $inputs.cityName
        successCriteria:
          - condition: $statusCode == 200
```

You Can't Improve What You Don't Measure

- Spectral & Co. aren't all you need
- *API Scorecard* scores your APIs across six dimensions
- Web: jentic.com/scorecard
- CLI: `npm install -g @jentic/api-scorecard-cli`
- Repo: github.com/jentic/jentic-api-scorecard



AI Readiness Score for APIs



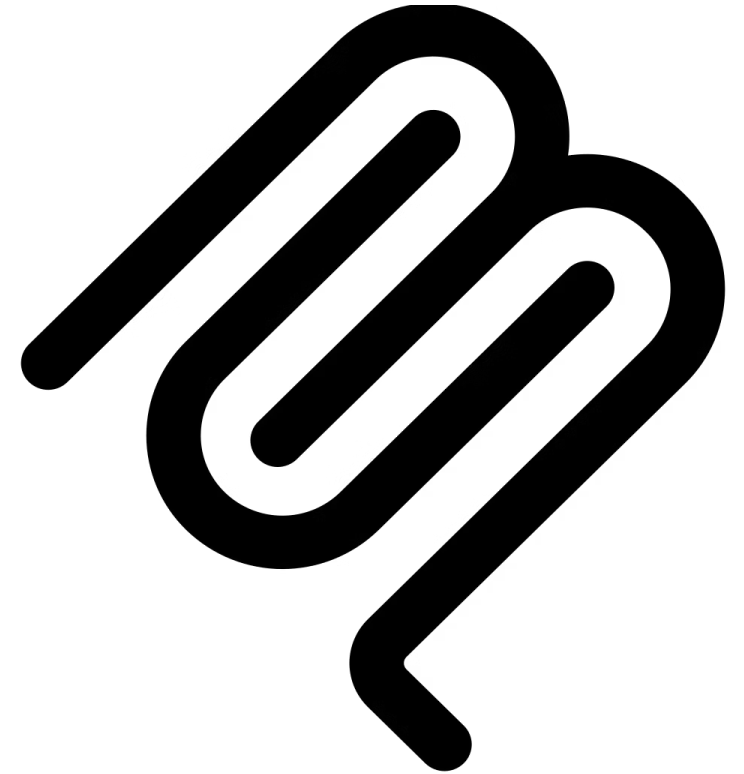
Outline

1. [Introduction](#) [3]
2. [API Design & Description in the Age of AI](#) [6]
3. [Exposing API Landscapes for AI Consumption](#) [9]
4. [MCP to the Rescue!?!](#) [2]
5. [Conclusions](#) [2]



Just Use MCP?

- The pitch: “APIs are old, MCP is the future”
- Wrap anything in MCP → call it AI-ready
- Skip the hard API work → straight to AI
- Just “start fresh” after 10 years of API landscape work?



MCP-Washing

- Skip the API work → jump to MCP → call it “AI-readiness”
- This is the wrong place to invest from a strategy perspective
- MCP should be a *thin* layer, not a *fat* one
- Invest in your APIs and then expose them through MCP



Outline

1. [Introduction](#) [3]
2. [API Design & Description in the Age of AI](#) [6]
3. [Exposing API Landscapes for AI Consumption](#) [9]
4. [MCP to the Rescue!?!](#) [2]
5. [Conclusions](#) [2]



Aligned with the iSAQB API Module

- Covered in the CPSA Advanced Level: API Module
 - API Design, Description, Lifecycle
 - API Security & APIs at Scale
- Background reading (iSAQB blog posts)
 - “APIs as Business Assets: Designing for Scale, Governance, and AI”
 - “APIs as Digital Building Blocks”



Thank You!

